Building Without Bytes

Orchestrating Containerized Workflows with Bazel

These slides were not exported from an app — they were **built by Bazel**, from the same workspace that builds the containers they describe.

This slide is a build artifact

`bazel build //slides:build` → this deck.
`bazel build //samples/python:image` → the image we'll discuss.

Same graph. Same cache. Same guarantees.

If you can model a talk as a build artifact,
you can model your whole supply chain as one.

The "before": an imperative Dockerfile

The Docker daemon installs the `dot` system dependency with `RUN apt-get` — an opaque, stateful layer. Editing `app.py` busts everything below the `COPY`.

```
# The "before": a traditional, imperative Dockerfile build (TASK §2.2).
#
# Note the system dependency handled the daemon way: `apt-get install graphviz`
# in a RUN layer. It works, but the layer is an opaque blob produced by a
# stateful daemon, and editing app.py below busts every layer after the COPY.
# Compare with //samples/python:image, where the same dependency is a
# declarative, content-addressed layer in the Bazel action graph.
#
# Build & run:
#   docker build -t bwb/python-graphviz-dockerfile samples/python
#   docker run --rm bwb/python-graphviz-dockerfile   # renders the PNG

FROM python:3.12-slim-bookworm

# System dependency: the `dot` binary. Imperative, daemon-run, cache-coarse.
RUN apt-get update \
    && apt-get install -y --no-install-recommends graphviz \
    && rm -rf /var/lib/apt/lists/*

# Python dependency layer.
COPY requirements_lock.txt /tmp/requirements.txt
```

The payload: Python + a *non-hermetic* system dep

`graphviz` shells out to the `dot` binary — the apt dependency we must make explicit once we leave the daemon. The app renders the workshop's own graph:

```
def build_graph() -> graphviz.Digraph:
    """The supply chain that builds this very talk (the dogfooding loop)."""
    dot = graphviz.Digraph("supply_chain", comment="Building Without Bytes")
    dot.attr(rankdir="LR")
    dot.attr("node", shape="box", style="rounded")

    dot.node("src", "source\n(app.py, BUILD)")
    dot.node("deps", "pip deps\n(graphviz)")
    dot.node("sysdep", "system layer\n(dot binary)", style="rounded,filled", fillcolor="lightyellow")
    dot.node("image", "oci_image")
    dot.node("slides", "slides deck")
    dot.node("registry", "registry")

    dot.edge("src", "image")
    dot.edge("deps", "image")
    dot.edge("sysdep", "image")
    dot.edge("image", "registry")
    dot.edge("src", "slides", label="transclude")  # the deck imports this file
    return dot
```

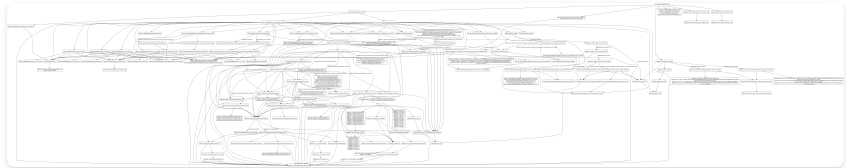
The "after": layers as declarative build outputs

No daemon. The image is composed from content-addressed layers in the action graph; changing `app.py` reuses the base + deps layers untouched.

```
oci_image(
    name = "image",
    base = "@python_base",
    entrypoint = [
        "/app/app",  # py_binary bootstrap launcher; locates /app/app.runfiles
        "--render",
        "/tmp/supply_chain.png",
    ],
    tars = [":app_layer"],
)
```

The dogfooding loop

The Python sample renders **this repo's own dependency graph** with `dot`, and the deck imports the result.



A Bazel action, needing an apt-installed binary packaged as a layer, draws the graph of the build that produced the slide.

```
bazel run //tools:render_graph \
  -- //samples/python:image
```

Takeaways

Slide-as-Code — clone this repo's structure.
Benchmark — measure Dockerfile vs. Bazel/OCI on your own images.
Mental model — layers are granular, cacheable build outputs, not blobs.

Once layers are just artifacts, the daemon stops being special.